# Response to the review of the Szöllősi-family MUB exclusion

30 September 2026

The review found exact counterexamples to three implementation contracts of the earlier version, an incomplete rounding argument in the parameter selector, and verification claims that overstated independence. It recommended major revision and set out what the work would need in order to reach the highest standard. This note records how each point was addressed. Section references are to the revised paper; file names are relative to the bundle.

## 1. What changed in the design

The central change is architectural. The earlier version asked the reader to trust a single program of about 1,700 lines of Python and NumPy, including its numerical solvers, its disc arithmetic and its BLAS calls. The revised version separates an *untrusted generator*, which produces certificates, from an *independent checker*, which verifies every certificate with no shared code (§8). The checker's floating-point work is done by a C kernel of 638 lines in which every result is moved one unit in the last place outward (Lemma 13); it uses no library transcendental function; it verifies the unit table itself with `mpmath` interval arithmetic; it re-derives the parameter cover with its own selector; it re-verifies by exhaustive branch and bound that each certificate's hulls contain all unbiased vectors; and it recomputes the orthogonality and unbiasedness graphs and the clique condition with its own code. All 14,658 Szöllősi certificates were regenerated with a repaired generator and replayed by the checker (75 of them on sub-boxes of their parameter box, which is valid because only the points of the fundamental domain matter; §6.5), and the checker's cover evaluation is complete (`chk_results/chk_szollosi_summary.json`).

To address transferability, the certificate format was made family-generic (§2: admissible families, certificates, proof obligations, Theorem 7), and the same generator and checker were applied to the Fourier family $F(a, b)$, giving a rigorous re-proof of Theorem 1.4 of Jaming et al. (2009) (§4, §9.3), after an exact proof that their fundamental triangle is a fundamental domain for a symmetry group of order 144 (Proposition 12).

## 2. Major comments

**4.1 (outward enclosure).** Both counterexamples were reproduced and are now regression tests (`tests/test_gen.py`). In the generator every radius passes through the correction of Lemma R (Appendix A), which is proved for monotone expressions of bounded length and depth; differences and lower bounds use single operations followed by one-ulp steps. More importantly, the proof no longer depends on the generator's arithmetic: the checker's kernel is tested against exact rational arithmetic on 16,052 enclosures including the referee's value $r = 1 + 2^{-52}$, and mutation testing shows that removing the outward step is detected (§8.3).

**4.2 (parameter selector).** The selector was rewritten twice, independently: in the generator and in the checker. Neither uses a library exponential or a floating-point angular constant; the ordering conditions are exact integer tests; the constraint and sector tests are evaluated in interval

arithmetic from the verified table, with an explicit bound for the support-function inequality that includes the norm of the candidate direction (§3.4, Lemma 11). The checker's selector is the one the proof uses.

**4.3 (independence).** Table 1 of the paper states, for each question the proof must answer, which program answers it and what that program shares with the generator. The only shared artefacts are the unit table (verified independently) and the certificates themselves (the data under verification). Boxes whose certificate has no hulls — previously opaque — are verified by the same branch and bound: every phase box is excluded (§6.4).

**4.4 (cyclic hull).** Repaired (wrapping intervals are split before merging) and tested on the referee's example and 3,000 adversarial cases; hulls are now formed from dyadic boxes only. A wrong hull could in any case only cause the checker to reject a certificate.

**4.5 (resumption).** Records have explicit states; only certified records are skipped on resumption; the referee's synthetic record is a regression test. The checker never trusts a record's status.

**4.6 (positioning; degeneracy).** The comparison with Jaming et al. was rewritten from their paper and program (§1.3, §11.2): their discretisation lemmas are explicit mathematics, their implementation's rounding control is a tolerance of $10^{-9}$ justified by a code comment, and their theorem is now re-proved rigorously. Statements about coalescing solutions are now numerical diagnostics in 60-digit arithmetic (§10), which leave open whether the singular solutions are isolated; "no isolation argument can succeed" is replaced by a precise statement about simple-root isolation at fixed resolution.

## 3. Minor comments

- Provenance: every raw results file, log and checker output is in the bundle, with SHA-256 checksums (`SHA256SUMS`, Appendix C).
- The solution-count histogram is complete and labelled as numerical (Table 2).
- No figure is offered as evidence of coverage; coverage is the checker's cover-tree evaluation.
- The exact ray point $\frac{1}{2} + \frac{i}{2\sqrt{3}}$ is used.
- Empty hulls: no inference about $P \cap \Sigma$ is made; such certificates are verified directly, and as a separate diagnostic the checker's selector proves $P \cap \mathcal{F} = \emptyset$ for both such boxes.
- Table precision and module-count wording are corrected; Appendix C lists every module with its size and checksum.

## 4. "Should" and "could" recommendations

- Dependency diagram: Figure 1 and Table 1.
- Adversarial tests: generator regressions for every counterexample; an eleven-group kernel test-suite run on three platforms (x86-64 Linux, ARM64 Linux, ARM64 macOS); mutation testing of the kernel (16 of 17 mutations detected; the survivor is discussed in §8.3).
- Reusable interface: §2 and the Fourier application.

## 5. What remains outside the proof

The trusted computing base (§8.1) still includes the Python interpreter and NumPy for integer control flow, the C compiler, `mpmath`, SymPy and the hardware's IEEE arithmetic, and the mathematics is not formalised in a proof assistant. One mutation of the kernel — removing a second-order term of the Krawczyk operator — is not detected by any test, because interval overestimation elsewhere compensates for it in practice; its necessity rests on the proof of Lemma 17 and on reading the kernel. These are stated in the paper rather than hidden.